

Package ‘ggalt’

May 12, 2026

Title Extra Coordinate Systems, 'Geoms', Statistical Transformations,
Scales and Fonts for 'ggplot2'

Version 0.8.0

Maintainer Bob Rudis <bob@rud.is>

Description A compendium of new geometries, coordinate systems, statistical transformations, scales and fonts for 'ggplot2', including splines, 1d and 2d densities, univariate average shifted histograms, formatters for ``bytes", a stat_stepribbon() function, and the 'StateFace' open source font 'ProPublica'. Further new functionality includes lollipop charts, dumbbell charts, the ability to encircle points and coordinate-system-based text annotations.

License MIT + file LICENSE

URL <https://github.com/hrbrmstr/ggalt>

BugReports <https://github.com/hrbrmstr/ggalt/issues>

Encoding UTF-8

Depends R (>= 4.1), ggplot2 (>= 4.0.0)

Suggests testthat, gridExtra, knitr, rmarkdown, ggthemes, reshape2,
MASS, dplyr

Imports utils, graphics, datasets, grDevices, RColorBrewer,
KernSmooth, scales, grid, gtable, ash, maps, extrafont, tibble

RoxygenNote 7.3.3

VignetteBuilder knitr

Collate 'annotate_textp.r' 'annotation_ticks.r' 'formatters.r'
'fortify.r' 'position-dodgev.R' 'geom_ash.r' 'geom_bkde.r'
'geom_bkde2d.r' 'geom_spikelines.R' 'geom_dumbbell.R'
'geom_cartogram.r' 'geom_encircle.r' 'geom_horizon.r'
'geom_lollipop.r' 'geom_table.r' 'geom_twoway_bar.r'
'geom_xspline.r' 'geom_xspline2.r' 'geom_uhar.r'
'stat-stepribbon.r' 'ggalt-package.r' 'grob_absolute.r'
'guide_axis.r' 'stateface.r' 'utils.r' 'zzz.r'

NeedsCompilation no

Author Bob Rudis [aut, cre] (ORCID: <<https://orcid.org/0000-0001-5670-2640>>),
Ben Bolker [aut, ctb] (Encircling & additional splines),
Ben Marwick [ctb] (General codebase cleanup),
Jan Schulz [aut, ctb] (Annotations),
Rosen Matev [ctb] (Original annotate_textp implementation on

stackoverflow),
 ProPublica [dtc] (StateFace font),
 Aditya Kothari [aut, ctb] (Core functionality of horizon plots),
 Ather [dtc] (Core functionality of horizon plots),
 Jonathan Sidi [aut, ctb] (Annotation ticks),
 Tarcisio Fedrizzi [ctb] (Bytes formatter)

Contents

ggalt-package	2
absoluteGrob	3
annotate_textp	4
annotation_ticks	5
byte_format	7
fortify.table	8
GeomTicks	8
geom_bkde	8
geom_bkde2d	12
geom_cartogram	15
geom_dumbbell	18
geom_encircle	20
geom_horizon	21
geom_lollipop	22
geom_spikelines	24
geom_stateface	27
geom_ubar	29
geom_xspline	31
geom_xspline2	35
load_stateface	37
position_dodgev	37
show_stateface	38
StatAsh	38
stat_ash	39
stat_stepribbon	41
Index	44

ggalt-package	<i>ggalt: Extra Coordinate Systems, 'Geoms', Statistical Transformations, Scales and Fonts for 'ggplot2'</i>
---------------	--

Description

A compendium of new geometries, coordinate systems, statistical transformations, scales and fonts for 'ggplot2', including splines, 1d and 2d densities, univariate average shifted histograms, formatters for "bytes", a stat_stepribbon() function, and the 'StateFace' open source font 'ProPublica'. Further new functionality includes lollipop charts, dumbbell charts, the ability to encircle points and coordinate-system-based text annotations.

Author(s)

Maintainer: Bob Rudis <bob@rud.is> ([ORCID](#))

Authors:

- Ben Bolker (Encircling & additional splines) [contributor]
- Jan Schulz (Annotations) [contributor]
- Aditya Kothari (Core functionality of horizon plots) [contributor]
- Jonathan Sidi (Annotation ticks) [contributor]

Other contributors:

- Ben Marwick (General codebase cleanup) [contributor]
- Rosen Matev (Original annotate_textp implementation on stackoverflow) [contributor]
- ProPublica (StateFace font) [data contributor]
- Ather (Core functionality of horizon plots) [data contributor]
- Tarcisio Fedrizzi (Bytes formatter) [contributor]

See Also

Useful links:

- <https://github.com/hrbrmstr/ggalt>
- Report bugs at <https://github.com/hrbrmstr/ggalt/issues>

absoluteGrob

Absolute grob

Description

This grob has fixed dimensions and position.

Usage

```
absoluteGrob(  
  grob,  
  width = NULL,  
  height = NULL,  
  xmin = NULL,  
  ymin = NULL,  
  vp = NULL  
)
```

Details

It's still experimental

 annotate_textp

Text annotations in plot coordinate system

Description

Annotates the plot with text. Compared to `annotate("text", ...)`, the placement of the annotations is specified in plot coordinates (from 0 to 1) instead of data coordinates.

Usage

```
annotate_textp(
  label,
  x,
  y,
  facets = NULL,
  hjust = 0,
  vjust = 0,
  color = "black",
  alpha = NA,
  family = theme_get()$text$family,
  size = theme_get()$text$size,
  fontface = 1,
  lineheight = 1,
  box_just = ifelse(c(x, y) < 0.5, 0, 1),
  margin = unit(size/2, "pt")
)
```

Arguments

label	text annotation to be placed on the plot
x, y	positions of the individual annotations, in plot coordinates (0..1) instead of data coordinates!
facets	facet positions of the individual annotations
hjust, vjust	horizontal and vertical justification of the text relative to the bounding box
color	alpha, family, size, fontface, lineheight font properties
alpha, family, size, fontface, lineheight	standard aesthetic customizations
box_just	placement of the bounding box for the text relative to x,y coordinates. Per default, the box is placed to the center of the plot. Be aware that parts of the box which are outside of the visible region of the plot will not be shown.
margin	margins of the bounding box

Examples

```
p <- ggplot(mtcars, aes(x = wt, y = mpg)) + geom_point()
p <- p + geom_smooth(method = "lm", se = FALSE)
p + annotate_textp(x = 0.9, y = 0.35, label="A relative linear\nrelationship", hjust=1, color="red")
```

annotation_ticks	<i>Annotation: tick marks</i>
------------------	-------------------------------

Description

This annotation adds tick marks to an axis

Usage

```
annotation_ticks(
  sides = "b",
  scale = "identity",
  scaled = TRUE,
  short = unit(0.1, "cm"),
  mid = unit(0.2, "cm"),
  long = unit(0.3, "cm"),
  colour = "black",
  size = 0.5,
  linetype = 1,
  alpha = 1,
  color = NULL,
  ticks_per_base = NULL,
  ...
)
```

Arguments

sides	a string that controls which sides of the plot the log ticks appear on. It can be set to a string containing any of "trbl", for top, right, bottom, and left.
scale	character, vector of type of scale attributed to each corresponding side, Default: 'identity'
scaled	is the data already log-scaled? This should be TRUE (default) when the data is already transformed with log10() or when using scale_y_log10(). It should be FALSE when using coord_transform(y = "log10").
short	a grid::unit() object specifying the length of the short tick marks
mid	a grid::unit() object specifying the length of the middle tick marks. In base 10, these are the "5" ticks.
long	a grid::unit() object specifying the length of the long tick marks. In base 10, these are the "1" (or "10") ticks.
colour	Colour of the tick marks.
size	[Deprecated]
linetype	Linetype of tick marks (solid, dashed, etc.)
alpha	The transparency of the tick marks.
color	An alias for colour.
ticks_per_base	integer, number of minor ticks between each pair of major ticks, Default: NULL
...	Other parameters passed on to the layer

Details

If scale is of length one it will be replicated to the number of sides given, but if the length of scale is larger than one it must match the number of sides given. If ticks_per_base is set to NULL the function infers the number of ticks per base to be the base of the scale - 1, for example log scale is base exp(1) and log10 and identity are base 10. If ticks_per_base is given it follows the same logic as scale.

Author(s)

Jonathan Sidi

Examples

```
p <- ggplot(msleep, aes(bodywt, brainwt)) + geom_point()

# Default behavior

# add identity scale minor ticks on y axis
p + annotation_ticks(sides = 'l')

# add identity scale minor ticks on x,y axis
p + annotation_ticks(sides = 'lb')

# Control number of minor ticks of each side independently

# add identity scale minor ticks on x,y axis
p + annotation_ticks(sides = 'lb', ticks_per_base = c(10,5))

# log10 scale
p1 <- p + scale_x_log10()

# add minor ticks on log10 scale
p1 + annotation_ticks(sides = 'b', scale = 'log10')

# add minor ticks on both scales
p1 + annotation_ticks(sides = 'lb', scale = c('identity','log10'))

# add minor ticks on both scales, but force x axis to be identity
p1 + annotation_ticks(sides = 'lb', scale = 'identity')

# log scale
p2 <- p + scale_x_continuous(trans = 'log')

# add minor ticks on log scale
p2 + annotation_ticks(sides = 'b', scale = 'log')

# add minor ticks on both scales
p2 + annotation_ticks(sides = 'lb', scale = c('identity','log'))

# add minor ticks on both scales, but force x axis to be identity
p2 + annotation_ticks(sides = 'lb', scale = 'identity')
```

byte_format

*Bytes formatter: convert to byte measurement and display symbol.***Description**

Bytes formatter: convert to byte measurement and display symbol.

Usage

```
byte_format(symbol = "auto", units = "binary", only_highest = TRUE)
```

```
Kb(x)
```

```
Mb(x)
```

```
Gb(x)
```

```
bytes(x, symbol = "auto", units = c("binary", "si"), only_highest = FALSE)
```

Arguments

symbol	byte symbol to use. If "auto" the symbol used will be determined by the maximum value of x. Valid symbols are "b", "K", "Mb", "Gb", "Tb", "Pb", "Eb", "Zb", and "Yb", along with their upper case equivalents and "iB" equivalents.
units	which unit base to use, "binary" (1024 base) or "si" (1000 base) for ISI units.
only_highest	Whether to use the unit of the highest number or each number uses its own unit.
x	a numeric vector to format

Value

a function with three parameters, x, a numeric vector that returns a character vector, symbol a single or a vector of byte symbol(s) (e.g. "Kb") desired and the measurement units (traditional binary or si for ISI metric units).

References

Units of Information (Wikipedia) : https://en.wikipedia.org/wiki/Units_of_information

Examples

```
byte_format()(sample(3000000000, 10))
bytes(sample(3000000000, 10))
Kb(sample(3000000000, 10))
Mb(sample(3000000000, 10))
Gb(sample(3000000000, 10))
```

fortify.table	<i>Fortify contingency tables</i>
---------------	-----------------------------------

Description

Fortify contingency tables

Usage

```
## S3 method for class 'table'  
fortify(model, data, ...)
```

Arguments

model	the contingency table
data	data (unused)
...	(unused)

GeomTicks	<i>Base ggproto classes for ggplot2</i>
-----------	---

Description

If you are creating a new geom, stat, position, or scale in another package, you'll need to extend from `ggplot2::Geom`, `ggplot2::Stat`, `ggplot2::Position`, or `ggplot2::Scale`.

See Also

[ggplot2-ggproto](#)

geom_bkde	<i>Display a smooth density estimate.</i>
-----------	---

Description

A kernel density estimate, useful for displaying the distribution of variables with underlying smoothness.

Usage

```
geom_bkde(
  mapping = NULL,
  data = NULL,
  stat = "bkde",
  position = "identity",
  bandwidth = NULL,
  range.x = NULL,
  na.rm = FALSE,
  show.legend = NA,
  inherit.aes = TRUE,
  ...
)

stat_bkde(
  mapping = NULL,
  data = NULL,
  geom = "area",
  position = "stack",
  kernel = "normal",
  canonical = FALSE,
  bandwidth = NULL,
  gridsize = 410,
  range.x = NULL,
  truncate = TRUE,
  na.rm = FALSE,
  show.legend = NA,
  inherit.aes = TRUE,
  ...
)
```

Arguments

mapping	Set of aesthetic mappings created by aes() . If specified and <code>inherit.aes = TRUE</code> (the default), it is combined with the default mapping at the top level of the plot. You must supply mapping if there is no plot mapping.
data	The data to be displayed in this layer. There are three options: If <code>NULL</code>, the default, the data is inherited from the plot data as specified in the call to ggplot(). A <code>data.frame</code>, or other object, will override the plot data. All objects will be fortified to produce a data frame. See fortify() for which variables will be created. A function will be called with a single argument, the plot data. The return value must be a <code>data.frame</code>, and will be used as the layer data. A function can be created from a formula (e.g. <code>~ head(.x, 10)</code>).
position	A position adjustment to use on the data for this layer. This can be used in various ways, including to prevent overplotting and improving the display. The <code>position</code> argument accepts the following: • The result of calling a position function, such as <code>position_jitter()</code>. This method allows for passing extra arguments to the position.

	• A string naming the position adjustment. To give the position as a string, strip the function name of the <code>position_</code> prefix. For example, to use <code>position_jitter()</code>, give the position as "jitter". • For more information and other ways to specify the position, see the layer position documentation.
<code>bandwidth</code>	the kernel bandwidth smoothing parameter. see bkde for details. If <code>NULL</code> , it will be computed for you but will most likely not yield optimal results.
<code>range.x</code>	vector containing the minimum and maximum values of <code>x</code> at which to compute the estimate. see bkde for details
<code>na.rm</code>	If <code>FALSE</code> , the default, missing values are removed with a warning. If <code>TRUE</code> , missing values are silently removed.
<code>show.legend</code>	logical. Should this layer be included in the legends? <code>NA</code> , the default, includes if any aesthetics are mapped. <code>FALSE</code> never includes, and <code>TRUE</code> always includes. It can also be a named logical vector to finely select the aesthetics to display. To include legend keys for all levels, even when no data exists, use <code>TRUE</code> . If <code>NA</code> , all levels are shown in legend, but unobserved levels are omitted.
<code>inherit.aes</code>	If <code>FALSE</code> , overrides the default aesthetics, rather than combining with them. This is most useful for helper functions that define both data and aesthetics and shouldn't inherit behaviour from the default plot specification, e.g. annotation_borders() .
<code>...</code>	Other arguments passed on to layer()'s <code>params</code> argument. These arguments broadly fall into one of 4 categories below. Notably, further arguments to the <code>position</code> argument, or aesthetics that are required can <i>not</i> be passed through <code>...</code>. Unknown arguments that are not part of the 4 categories below are ignored. • Static aesthetics that are not mapped to a scale, but are at a fixed value and apply to the layer as a whole. For example, <code>colour = "red"</code> or <code>linewidth = 3</code>. The geom's documentation has an Aesthetics section that lists the available options. The 'required' aesthetics cannot be passed on to the <code>params</code>. Please note that while passing unmapped aesthetics as vectors is technically possible, the order and required length is not guaranteed to be parallel to the input data. • When constructing a layer using a <code>stat_*()</code> function, the <code>...</code> argument can be used to pass on parameters to the geom part of the layer. An example of this is <code>stat_density(geom = "area", outline.type = "both")</code>. The geom's documentation lists which parameters it can accept. • Inversely, when constructing a layer using a <code>geom_*()</code> function, the <code>...</code> argument can be used to pass on parameters to the stat part of the layer. An example of this is <code>geom_area(stat = "density", adjust = 0.5)</code>. The stat's documentation lists which parameters it can accept. • The <code>key_glyph</code> argument of layer() may also be passed on through <code>...</code>. This can be one of the functions described as key glyphs, to change the display of the layer in the legend.
<code>geom, stat</code>	Use to override the default connection between <code>geom_bkde</code> and <code>stat_bkde</code> .
<code>kernel</code>	character string which determines the smoothing kernel. see bkde for details
<code>canonical</code>	logical flag: if <code>TRUE</code> , canonically scaled kernels are used. see bkde for details
<code>gridsize</code>	the number of equally spaced points at which to estimate the density. see bkde for details.
<code>truncate</code>	logical flag: if <code>TRUE</code> , data with <code>x</code> values outside the range specified by <code>range.x</code> are ignored. see bkde for details

Details

A sample of the output from `geom_bkde()`:

Aesthetics

`geom_bkde` understands the following aesthetics (required aesthetics are in bold):

- **x**
- **y**
- **alpha**
- **color**
- **fill**
- **linetype**
- **size**

Computed variables

density density estimate

count density * number of points - useful for stacked density plots

scaled density estimate, scaled to maximum of 1

See Also

See [geom_histogram](#), [geom_freqpoly](#) for other methods of displaying continuous distribution.

See [geom_violin](#) for a compact density display.

Examples

```
data(geyser, package="MASS")

ggplot(geyser, aes(x=duration)) +
  stat_bkde(alpha=1/2)

ggplot(geyser, aes(x=duration)) +
  geom_bkde(alpha=1/2)

ggplot(geyser, aes(x=duration)) +
  stat_bkde(bandwidth=0.25)

ggplot(geyser, aes(x=duration)) +
  geom_bkde(bandwidth=0.25)
```

geom_bkde2d

*Contours from a 2d density estimate.***Description**

Perform a 2D kernel density estimation using `bkde2D` and display the results with contours. This can be useful for dealing with overplotting

Usage

```
geom_bkde2d(
  mapping = NULL,
  data = NULL,
  stat = "bkde2d",
  position = "identity",
  bandwidth = NULL,
  range.x = NULL,
  lineend = "butt",
  contour = TRUE,
  linejoin = "round",
  linemitre = 1,
  na.rm = FALSE,
  show.legend = NA,
  inherit.aes = TRUE,
  ...
)

stat_bkde2d(
  mapping = NULL,
  data = NULL,
  geom = "density2d",
  position = "identity",
  contour = TRUE,
  bandwidth = NULL,
  grid_size = c(51, 51),
  range.x = NULL,
  truncate = TRUE,
  na.rm = FALSE,
  show.legend = NA,
  inherit.aes = TRUE,
  ...
)
```

Arguments

mapping	Set of aesthetic mappings created by <code>aes()</code> . If specified and <code>inherit.aes = TRUE</code> (the default), it is combined with the default mapping at the top level of the plot. You must supply mapping if there is no plot mapping.
data	The data to be displayed in this layer. There are three options: If <code>NULL</code> , the default, the data is inherited from the plot data as specified in the call to <code>ggplot()</code> .

A `data.frame`, or other object, will override the plot data. All objects will be fortified to produce a data frame. See [fortify\(\)](#) for which variables will be created.

A function will be called with a single argument, the plot data. The return value must be a `data.frame`, and will be used as the layer data. A function can be created from a formula (e.g. `~ head(.x, 10)`).

stat

The statistical transformation to use on the data for this layer. When using a `geom_*()` function to construct a layer, the `stat` argument can be used to override the default coupling between geoms and stats. The `stat` argument accepts the following:

- A `Stat` ggproto subclass, for example `StatCount`.
- A string naming the stat. To give the stat as a string, strip the function name of the `stat_` prefix. For example, to use `stat_count()`, give the stat as "count".
- For more information and other ways to specify the stat, see the [layer stat](#) documentation.

position

A position adjustment to use on the data for this layer. This can be used in various ways, including to prevent overplotting and improving the display. The `position` argument accepts the following:

- The result of calling a position function, such as `position_jitter()`. This method allows for passing extra arguments to the position.
- A string naming the position adjustment. To give the position as a string, strip the function name of the `position_` prefix. For example, to use `position_jitter()`, give the position as "jitter".
- For more information and other ways to specify the position, see the [layer position](#) documentation.

bandwidth

the kernel bandwidth smoothing parameter. see [bkde2D](#) for details. If `NULL`, it will be computed for you but will most likely not yield optimal results. see [bkde2D](#) for details

range.x

a list containing two vectors, where each vector contains the minimum and maximum values of `x` at which to compute the estimate for each direction. see [bkde2D](#) for details

lineend

Line end style (round, butt, square).

contour

If `TRUE`, contour the results of the 2d density estimation

linejoin

Line join style (round, mitre, bevel).

linemitre

Line mitre limit (number greater than 1).

na.rm

If `FALSE`, the default, missing values are removed with a warning. If `TRUE`, missing values are silently removed.

show.legend

logical. Should this layer be included in the legends? `NA`, the default, includes if any aesthetics are mapped. `FALSE` never includes, and `TRUE` always includes. It can also be a named logical vector to finely select the aesthetics to display. To include legend keys for all levels, even when no data exists, use `TRUE`. If `NA`, all levels are shown in legend, but unobserved levels are omitted.

inherit.aes

If `FALSE`, overrides the default aesthetics, rather than combining with them. This is most useful for helper functions that define both data and aesthetics and shouldn't inherit behaviour from the default plot specification, e.g. [annotation_borders\(\)](#).

...	Other arguments passed on to <code>layer()</code> 's <code>params</code> argument. These arguments broadly fall into one of 4 categories below. Notably, further arguments to the <code>position</code> argument, or aesthetics that are required can <i>not</i> be passed through ... Unknown arguments that are not part of the 4 categories below are ignored.
	• Static aesthetics that are not mapped to a scale, but are at a fixed value and apply to the layer as a whole. For example, <code>colour = "red"</code> or <code>linewidth = 3</code>. The geom's documentation has an Aesthetics section that lists the available options. The 'required' aesthetics cannot be passed on to the <code>params</code>. Please note that while passing unmapped aesthetics as vectors is technically possible, the order and required length is not guaranteed to be parallel to the input data. • When constructing a layer using a <code>stat_*()</code> function, the ... argument can be used to pass on parameters to the geom part of the layer. An example of this is <code>stat_density(geom = "area", outline.type = "both")</code>. The geom's documentation lists which parameters it can accept. • Inversely, when constructing a layer using a <code>geom_*()</code> function, the ... argument can be used to pass on parameters to the stat part of the layer. An example of this is <code>geom_area(stat = "density", adjust = 0.5)</code>. The stat's documentation lists which parameters it can accept. • The <code>key_glyph</code> argument of <code>layer()</code> may also be passed on through ... This can be one of the functions described as key glyphs, to change the display of the layer in the legend.
<code>geom</code>	default geom to use with this stat
<code>grid_size</code>	vector containing the number of equally spaced points in each direction over which the density is to be estimated. see bkde2D for details
<code>truncate</code>	logical flag: if TRUE, data with x values outside the range specified by <code>range.x</code> are ignored. see bkde2D for details

Details

A sample of the output from `geom_bkde2d()`:

Computed variables

Same as [stat_contour](#)

See Also

[geom_contour](#) for contour drawing geom, [stat_sum](#) for another way of dealing with overplotting

Examples

```
m <- ggplot(faithful, aes(x = eruptions, y = waiting)) +
  geom_point() +
  xlim(0.5, 6) +
  ylim(40, 110)

m + geom_bkde2d(bandwidth=c(0.5, 4))

m + stat_bkde2d(bandwidth=c(0.5, 4), aes(fill = ..level..), geom = "polygon")

# If you map an aesthetic to a categorical variable, you will get a
# set of contours for each value of that variable
```

```

set.seed(4393)
dsmall <- diamonds[sample(nrow(diamonds), 1000), ]
d <- ggplot(dsmall, aes(x, y)) +
  geom_bkde2d(bandwidth=c(0.5, 0.5), aes(colour = cut))
d

# If we turn contouring off, we can use use geoms like tiles:
d + stat_bkde2d(bandwidth=c(0.5, 0.5), geom = "raster",
  aes(fill = ..density..), contour = FALSE)

# Or points:
d + stat_bkde2d(bandwidth=c(0.5, 0.5), geom = "point",
  aes(size = ..density..), contour = FALSE)

```

geom_cartogram	<i>Map polygons layer enabling the display of show statistical information</i>
----------------	--

Description

This replicates the old behaviour of `geom_map()`, enabling specifying of x and y aesthetics.

Usage

```

geom_cartogram(
  mapping = NULL,
  data = NULL,
  stat = "identity",
  ...,
  map,
  na.rm = FALSE,
  show.legend = NA,
  inherit.aes = TRUE
)

```

Arguments

mapping	Set of aesthetic mappings created by <code>aes()</code> . If specified and <code>inherit.aes = TRUE</code> (the default), it is combined with the default mapping at the top level of the plot. You must supply mapping if there is no plot mapping.
data	The data to be displayed in this layer. There are three options: If <code>NULL</code>, the default, the data is inherited from the plot data as specified in the call to <code>ggplot()</code>. A <code>data.frame</code>, or other object, will override the plot data. All objects will be fortified to produce a data frame. See <code>fortify()</code> for which variables will be created. A function will be called with a single argument, the plot data. The return value must be a <code>data.frame</code>, and will be used as the layer data. A function can be created from a formula (e.g. <code>~ head(.x, 10)</code>).

stat	The statistical transformation to use on the data for this layer. When using a <code>geom_*()</code> function to construct a layer, the <code>stat</code> argument can be used to override the default coupling between geoms and stats. The <code>stat</code> argument accepts the following: • A Stat ggproto subclass, for example <code>StatCount</code>. • A string naming the stat. To give the stat as a string, strip the function name of the <code>stat_</code> prefix. For example, to use <code>stat_count()</code>, give the stat as <code>"count"</code>. • For more information and other ways to specify the stat, see the layer stat documentation.
...	Other arguments passed on to <code>layer()</code>'s <code>params</code> argument. These arguments broadly fall into one of 4 categories below. Notably, further arguments to the <code>position</code> argument, or aesthetics that are required can <i>not</i> be passed through ... Unknown arguments that are not part of the 4 categories below are ignored. • Static aesthetics that are not mapped to a scale, but are at a fixed value and apply to the layer as a whole. For example, <code>colour = "red"</code> or <code>linewidth = 3</code>. The geom's documentation has an Aesthetics section that lists the available options. The 'required' aesthetics cannot be passed on to the <code>params</code>. Please note that while passing unmapped aesthetics as vectors is technically possible, the order and required length is not guaranteed to be parallel to the input data. • When constructing a layer using a <code>stat_*()</code> function, the ... argument can be used to pass on parameters to the geom part of the layer. An example of this is <code>stat_density(geom = "area", outline.type = "both")</code>. The geom's documentation lists which parameters it can accept. • Inversely, when constructing a layer using a <code>geom_*()</code> function, the ... argument can be used to pass on parameters to the stat part of the layer. An example of this is <code>geom_area(stat = "density", adjust = 0.5)</code>. The stat's documentation lists which parameters it can accept. • The <code>key_glyph</code> argument of <code>layer()</code> may also be passed on through ... This can be one of the functions described as key glyphs, to change the display of the layer in the legend.
map	Data frame that contains the map coordinates. This will typically be created using fortify on a spatial object. It must contain columns <code>x</code> , <code>long</code> or <code>longitude</code> , <code>y</code> , <code>lat</code> or <code>latitude</code> and <code>region</code> or <code>id</code> .
na.rm	If FALSE, the default, missing values are removed with a warning. If TRUE, missing values are silently removed.
show.legend	logical. Should this layer be included in the legends? NA, the default, includes if any aesthetics are mapped. FALSE never includes, and TRUE always includes. It can also be a named logical vector to finely select the aesthetics to display. To include legend keys for all levels, even when no data exists, use TRUE. If NA, all levels are shown in legend, but unobserved levels are omitted.
inherit.aes	If FALSE, overrides the default aesthetics, rather than combining with them. This is most useful for helper functions that define both data and aesthetics and shouldn't inherit behaviour from the default plot specification, e.g. annotation_borders() .

Aesthetics

`geom_cartogram` understands the following aesthetics (required aesthetics are in bold):

- `map_id`

- alpha
- colour
- fill
- group
- linetype
- size
- x
- y

Examples

```
## Not run:
# When using geom_polygon, you will typically need two data frames:
# one contains the coordinates of each polygon (positions), and the
# other the values associated with each polygon (values). An id
# variable links the two together

ids <- factor(c("1.1", "2.1", "1.2", "2.2", "1.3", "2.3"))

values <- data.frame(
  id = ids,
  value = c(3, 3.1, 3.1, 3.2, 3.15, 3.5)
)

positions <- data.frame(
  id = rep(ids, each = 4),
  x = c(2, 1, 1.1, 2.2, 1, 0, 0.3, 1.1, 2.2, 1.1, 1.2, 2.5, 1.1, 0.3,
        0.5, 1.2, 2.5, 1.2, 1.3, 2.7, 1.2, 0.5, 0.6, 1.3),
  y = c(-0.5, 0, 1, 0.5, 0, 0.5, 1.5, 1, 0.5, 1, 2.1, 1.7, 1, 1.5,
        2.2, 2.1, 1.7, 2.1, 3.2, 2.8, 2.1, 2.2, 3.3, 3.2)
)

ggplot() +
  geom_cartogram(aes(x, y, map_id = id), map = positions, data=positions)

ggplot() +
  geom_cartogram(aes(x, y, map_id = id), map = positions, data=positions) +
  geom_cartogram(data=values, map=positions, aes(fill = value, map_id=id))

ggplot() +
  geom_cartogram(aes(x, y, map_id = id), map = positions, data=positions) +
  geom_cartogram(data=values, map=positions, aes(fill = value, map_id=id)) +
  ylim(0, 3)

# Better example
crimes <- data.frame(state = tolower(rownames(USArrests)), USArrests)
crimesm <- reshape2::melt(crimes, id = 1)

if (require(maps)) {
  states_map <- map_data("state")

  ggplot() +
    geom_cartogram(aes(long, lat, map_id = region), map = states_map, data=states_map) +
```

```

geom_cartogram(aes(fill = Murder, map_id = state), map=states_map, data=crimes)

last_plot() + coord_map("polyconic")

ggplot() +
  geom_cartogram(aes(long, lat, map_id=region), map = states_map, data=states_map) +
  geom_cartogram(aes(fill = value, map_id=state), map = states_map, data=crimesm) +
  coord_map("polyconic") +
  facet_wrap( ~ variable)
}

## End(Not run)

```

geom_dumbbell

Dumbbell charts

Description

The dumbbell geom is used to create dumbbell charts.

Usage

```

geom_dumbbell(
  mapping = NULL,
  data = NULL,
  ...,
  colour_x = NULL,
  size_x = NULL,
  colour_xend = NULL,
  size_xend = NULL,
  dot_guide = FALSE,
  dot_guide_size = NULL,
  dot_guide_colour = NULL,
  na.rm = FALSE,
  show.legend = NA,
  inherit.aes = TRUE,
  position = "identity"
)

```

Arguments

mapping	Set of aesthetic mappings created by aes() . If specified and <code>inherit.aes = TRUE</code> (the default), it is combined with the default mapping at the top level of the plot. You must supply mapping if there is no plot mapping.
data	The data to be displayed in this layer. There are three options: If <code>NULL</code>, the default, the data is inherited from the plot data as specified in the call to ggplot(). A <code>data.frame</code>, or other object, will override the plot data. All objects will be fortified to produce a data frame. See fortify() for which variables will be created. A function will be called with a single argument, the plot data. The return value must be a <code>data.frame</code>, and will be used as the layer data. A function can be created from a formula (e.g. <code>~ head(.x, 10)</code>).

...	other arguments passed on to layer . These are often aesthetics, used to set an aesthetic to a fixed value, like <code>color = "red"</code> or <code>size = 3</code> . They may also be parameters to the paired geom/stat.
<code>colour_x</code>	the colour of the start point
<code>size_x</code>	the size of the start point
<code>colour_xend</code>	the colour of the end point
<code>size_xend</code>	the size of the end point
<code>dot_guide</code>	if TRUE, a leading dotted line will be placed before the left-most dumbbell point
<code>dot_guide_size, dot_guide_colour</code>	single-value aesthetics for <code>dot_guide</code>
<code>na.rm</code>	If FALSE (the default), removes missing values with a warning. If TRUE silently removes missing values.
<code>show.legend</code>	logical. Should this layer be included in the legends? NA, the default, includes if any aesthetics are mapped. FALSE never includes, and TRUE always includes. It can also be a named logical vector to finely select the aesthetics to display. To include legend keys for all levels, even when no data exists, use TRUE. If NA, all levels are shown in legend, but unobserved levels are omitted.
<code>inherit.aes</code>	If FALSE, overrides the default aesthetics, rather than combining with them. This is most useful for helper functions that define both data and aesthetics and shouldn't inherit behaviour from the default plot specification, e.g. annotation_borders() .
<code>position</code>	Position adjustment, either as a string, or the result of a call to a position adjustment function.

Details

Dumbbell dot plots — dot plots with two or more series of data — are an alternative to the clustered bar chart or slope graph.

Aesthetics

@section Aesthetics: `geom_segment()` understands the following aesthetics. Required aesthetics are displayed in bold and defaults are displayed for optional aesthetics:

- `x`
- `y`
- `xend`**`yend`**
- `alpha` → NA
- `colour` → via `theme()`
- `group` → inferred
- `linetype` → via `theme()`
- `linewidth` → via `theme()`

Learn more about setting these aesthetics in `vignette("ggplot2-specs")`.

Examples

```
library(ggplot2)
```

```
df <- data.frame(trt=LETTERS[1:5], l=c(20, 40, 10, 30, 50), r=c(70, 50, 30, 60, 80))
```

```

ggplot(df, aes(y=trt, x=l, xend=r)) +
  geom_dumbbell(size=3, color="#e3e2e1",
               colour_x = "#5b8124", colour_xend = "#bad744",
               dot_guide=TRUE, dot_guide_size=0.25) +
  labs(x=NULL, y=NULL, title="ggplot2 geom_dumbbell with dot guide") +
  theme_minimal() +
  theme(panel.grid.major.x=element_line(size=0.05))

## with vertical dodging
df2 <- data.frame(trt = c(LETTERS[1:5], "D"),
                  l = c(20, 40, 10, 30, 50, 40),
                  r = c(70, 50, 30, 60, 80, 70))

ggplot(df2, aes(y=trt, x=l, xend=r)) +
  geom_dumbbell(size=3, color="#e3e2e1",
               colour_x = "#5b8124", colour_xend = "#bad744",
               dot_guide=TRUE, dot_guide_size=0.25,
               position=position_dodgev(height=0.4)) +
  labs(x=NULL, y=NULL, title="ggplot2 geom_dumbbell with dot guide") +
  theme_minimal() +
  theme(panel.grid.major.x=element_line(size=0.05))

```

geom_encircle

Automatically enclose points in a polygon

Description

Automatically enclose points in a polygon

Usage

```

geom_encircle(
  mapping = NULL,
  data = NULL,
  stat = "identity",
  position = "identity",
  na.rm = FALSE,
  show.legend = NA,
  inherit.aes = TRUE,
  ...
)

```

Arguments

mapping	mapping
data	data
stat	stat
position	position
na.rm	na.rm
show.legend	show.legend
inherit.aes	inherit.aes
...	dots

Details

A sample of the output from `geom_encircle()`:

Value

adds a circle around the specified points

Author(s)

Ben Bolker

Examples

```
d <- data.frame(x=c(1,1,2),y=c(1,2,2)*100)

gg <- ggplot(d,aes(x,y))
gg <- gg + scale_x_continuous(expand=c(0.5,1))
gg <- gg + scale_y_continuous(expand=c(0.5,1))

gg + geom_encircle(s_shape=1, expand=0) + geom_point()

gg + geom_encircle(s_shape=1, expand=0.1, colour="red") + geom_point()

gg + geom_encircle(s_shape=0.5, expand=0.1, colour="purple") + geom_point()

gg + geom_encircle(data=subset(d, x==1), colour="blue", spread=0.02) +
  geom_point()

gg + geom_encircle(data=subset(d, x==2), colour="cyan", spread=0.04) +
  geom_point()

gg <- ggplot(mpg, aes(displ, hwy))
gg + geom_encircle(data=subset(mpg, hwy>40)) + geom_point()
gg + geom_encircle(aes(group=manufacturer)) + geom_point()
gg + geom_encircle(aes(group=manufacturer,fill=manufacturer),alpha=0.4)+
  geom_point()
gg + geom_encircle(aes(group=manufacturer,colour=manufacturer))+
  geom_point()

ss <- subset(mpg,hwy>31 & displ<2)

gg + geom_encircle(data=ss, colour="blue", s_shape=0.9, expand=0.07) +
  geom_point() + geom_point(data=ss, colour="blue")
```

geom_horizon

Plot a time series as a horizon plot

Description

A horizon plot breaks the Y dimension down using colours. This is useful when visualising y values spanning a vast range and / or trying to highlight outliers without losing context of the rest of the data.

Horizon plots are best viewed in an aspect ratio of very low vertical length.

Usage

```
geom_horizon(
  mapping = NULL,
  data = NULL,
  show.legend = TRUE,
  inherit.aes = TRUE,
  na.rm = TRUE,
  bandwidth = NULL,
  ...
)
```

GeomHorizon

```
stat_horizon(
  mapping = NULL,
  data = NULL,
  geom = "horizon",
  show.legend = TRUE,
  inherit.aes = TRUE,
  na.rm = TRUE,
  bandwidth = NULL,
  ...
)
```

StatHorizon

Format

An object of class GeomHorizon (inherits from GeomArea, GeomRibbon, Geom, ggproto, gg) of length 4.

An object of class StatHorizon (inherits from Stat, ggproto, gg) of length 5.

Aesthetics

x, y, fill. fill defaults to ..band.. which is the band number the current data fill area belongs in.

Other parameters

bandwidth, to dictate the span of a band.

geom_lollipop

Lollipop charts

Description

The lollipop geom is used to create lollipop charts.

Usage

```
geom_lollipop(
  mapping = NULL,
  data = NULL,
  ...,
  horizontal = FALSE,
  point.colour = NULL,
  point.size = NULL,
  na.rm = FALSE,
  show.legend = NA,
  inherit.aes = TRUE
)
```

Arguments

mapping	Set of aesthetic mappings created by aes() . If specified and <code>inherit.aes = TRUE</code> (the default), it is combined with the default mapping at the top level of the plot. You must supply mapping if there is no plot mapping.
data	The data to be displayed in this layer. There are three options: If <code>NULL</code>, the default, the data is inherited from the plot data as specified in the call to ggplot(). A <code>data.frame</code>, or other object, will override the plot data. All objects will be fortified to produce a data frame. See fortify() for which variables will be created. A function will be called with a single argument, the plot data. The return value must be a <code>data.frame</code>, and will be used as the layer data. A function can be created from a formula (e.g. <code>~ head(.x, 10)</code>).
...	other arguments passed on to layer . These are often aesthetics, used to set an aesthetic to a fixed value, like <code>color = "red"</code> or <code>size = 3</code> . They may also be parameters to the paired geom/stat.
horizontal	<code>horizontal</code> is <code>FALSE</code> (the default), the function will draw the lollipops up from the X axis (i.e. it will set <code>xend</code> to <code>x</code> & <code>yend</code> to <code>0</code>). If <code>TRUE</code> , it will set <code>yend</code> to <code>y</code> & <code>xend</code> to <code>0</code> . Make sure you map the <code>x</code> & <code>y</code> aesthetics accordingly. This parameter helps avoid the need for coord_flip() .
point.colour	the colour of the point
point.size	the size of the point
na.rm	If <code>FALSE</code> (the default), removes missing values with a warning. If <code>TRUE</code> silently removes missing values.
show.legend	logical. Should this layer be included in the legends? <code>NA</code> , the default, includes if any aesthetics are mapped. <code>FALSE</code> never includes, and <code>TRUE</code> always includes. It can also be a named logical vector to finely select the aesthetics to display. To include legend keys for all levels, even when no data exists, use <code>TRUE</code> . If <code>NA</code> , all levels are shown in legend, but unobserved levels are omitted.
inherit.aes	If <code>FALSE</code> , overrides the default aesthetics, rather than combining with them. This is most useful for helper functions that define both data and aesthetics and shouldn't inherit behaviour from the default plot specification, e.g. annotation_borders() .

Details

Lollipop charts are the creation of Andy Cotgreave going back to 2011. They are a combination of a thin segment, starting at with a dot at the top and are a suitable alternative to or replacement for bar charts.

Use the `horizontal` parameter to abate the need for `coord_flip()` (see the Arguments section for details).

A sample of the output from `geom_lollipop()`:

Aesthetics

@section Aesthetics: `geom_point()` understands the following aesthetics. Required aesthetics are displayed in bold and defaults are displayed for optional aesthetics:

- `x`
- `y`
- `alpha` → NA
- `colour` → via `theme()`
- `fill` → via `theme()`
- `group` → inferred
- `shape` → via `theme()`
- `size` → via `theme()`
- `stroke` → via `theme()`

Learn more about setting these aesthetics in `vignette("ggplot2-specs")`.

Examples

```
df <- data.frame(trt=LETTERS[1:10],
                 value=seq(100, 10, by=-10))

ggplot(df, aes(trt, value)) + geom_lollipop()

ggplot(df, aes(value, trt)) + geom_lollipop(horizontal=TRUE)
```

geom_spikelines

Draw spikelines on a plot

Description

Segment reference lines that originate at an point

Usage

```
geom_spikelines(
  mapping = NULL,
  data = NULL,
  stat = "identity",
  position = "identity",
  ...,

```



```

    arrow = NULL,
    lineend = "butt",
    linejoin = "round",
    na.rm = FALSE,
    show.legend = NA,
    inherit.aes = TRUE
  )

```

Arguments

mapping	Set of aesthetic mappings created by aes() . If specified and <code>inherit.aes = TRUE</code> (the default), it is combined with the default mapping at the top level of the plot. You must supply mapping if there is no plot mapping.
data	The data to be displayed in this layer. There are three options: If <code>NULL</code>, the default, the data is inherited from the plot data as specified in the call to ggplot(). A <code>data.frame</code>, or other object, will override the plot data. All objects will be fortified to produce a data frame. See fortify() for which variables will be created. A function will be called with a single argument, the plot data. The return value must be a <code>data.frame</code>, and will be used as the layer data. A function can be created from a formula (e.g. <code>~ head(.x, 10)</code>).
stat	The statistical transformation to use on the data for this layer. When using a <code>geom_*()</code> function to construct a layer, the <code>stat</code> argument can be used to override the default coupling between geoms and stats. The <code>stat</code> argument accepts the following: • A Stat ggproto subclass, for example <code>StatCount</code>. • A string naming the stat. To give the stat as a string, strip the function name of the <code>stat_</code> prefix. For example, to use <code>stat_count()</code>, give the stat as <code>"count"</code>. • For more information and other ways to specify the stat, see the layer stat documentation.
position	A position adjustment to use on the data for this layer. This can be used in various ways, including to prevent overplotting and improving the display. The <code>position</code> argument accepts the following: • The result of calling a position function, such as <code>position_jitter()</code>. This method allows for passing extra arguments to the position. • A string naming the position adjustment. To give the position as a string, strip the function name of the <code>position_</code> prefix. For example, to use <code>position_jitter()</code>, give the position as <code>"jitter"</code>. • For more information and other ways to specify the position, see the layer position documentation.
...	Other arguments passed on to layer()'s <code>params</code> argument. These arguments broadly fall into one of 4 categories below. Notably, further arguments to the <code>position</code> argument, or aesthetics that are required can <i>not</i> be passed through Unknown arguments that are not part of the 4 categories below are ignored. • Static aesthetics that are not mapped to a scale, but are at a fixed value and apply to the layer as a whole. For example, <code>colour = "red"</code> or <code>linewidth = 3</code>. The geom's documentation has an Aesthetics section that lists the available options. The 'required' aesthetics cannot be passed on to the

params. Please note that while passing unmapped aesthetics as vectors is technically possible, the order and required length is not guaranteed to be parallel to the input data.

- When constructing a layer using a `stat_*()` function, the `...` argument can be used to pass on parameters to the geom part of the layer. An example of this is `stat_density(geom = "area", outline.type = "both")`. The geom's documentation lists which parameters it can accept.
- Inversely, when constructing a layer using a `geom_*()` function, the `...` argument can be used to pass on parameters to the stat part of the layer. An example of this is `geom_area(stat = "density", adjust = 0.5)`. The stat's documentation lists which parameters it can accept.
- The `key_glyph` argument of `layer()` may also be passed on through `...`. This can be one of the functions described as [key glyphs](#), to change the display of the layer in the legend.

<code>arrow</code>	Arrow specification, as created by <code>grid::arrow()</code> .
<code>lineend</code>	Line end style (round, butt, square).
<code>linejoin</code>	Line join style (round, mitre, bevel).
<code>na.rm</code>	If FALSE, the default, missing values are removed with a warning. If TRUE, missing values are silently removed.
<code>show.legend</code>	logical. Should this layer be included in the legends? NA, the default, includes if any aesthetics are mapped. FALSE never includes, and TRUE always includes. It can also be a named logical vector to finely select the aesthetics to display. To include legend keys for all levels, even when no data exists, use TRUE. If NA, all levels are shown in legend, but unobserved levels are omitted.
<code>inherit.aes</code>	If FALSE, overrides the default aesthetics, rather than combining with them. This is most useful for helper functions that define both data and aesthetics and shouldn't inherit behaviour from the default plot specification, e.g. <code>annotation_borders()</code> .

Author(s)

Jonathan Sidi

Examples

```
mtcars$name <- rownames(mtcars)

p <- ggplot(data = mtcars, aes(x=mpg,y=disp)) + geom_point()

p + geom_spikelines(data = mtcars[mtcars$carb==4,], linetype = 2)

p + geom_spikelines(data = mtcars[mtcars$carb==4,], aes(colour = factor(gear)), linetype = 2)

## Not run:
require(ggrepel)
p + geom_spikelines(data = mtcars[mtcars$carb==4,], aes(colour = factor(gear)), linetype = 2) +
ggrepel::geom_label_repel(data = mtcars[mtcars$carb==4,], aes(label = name))

## End(Not run)
```

geom_stateface

*Use ProPublica's StateFace font in ggplot2 plots***Description**

The `label` parameter can be either a 2-letter state abbreviation or a full state name. `geom_stateface()` will take care of the translation to StateFace font glyph characters.

Usage

```
geom_stateface(
  mapping = NULL,
  data = NULL,
  stat = "identity",
  position = "identity",
  ...,
  parse = FALSE,
  nudge_x = 0,
  nudge_y = 0,
  check_overlap = FALSE,
  na.rm = FALSE,
  show.legend = NA,
  inherit.aes = TRUE
)
```

Arguments

- | | |
|---------|--|
| mapping | Set of aesthetic mappings created by aes() . If specified and <code>inherit.aes = TRUE</code> (the default), it is combined with the default mapping at the top level of the plot. You must supply <code>mapping</code> if there is no plot mapping. |
| data | <p>The data to be displayed in this layer. There are three options:</p> <p>If <code>NULL</code>, the default, the data is inherited from the plot data as specified in the call to ggplot().</p> <p>A <code>data.frame</code>, or other object, will override the plot data. All objects will be fortified to produce a data frame. See fortify() for which variables will be created.</p> <p>A function will be called with a single argument, the plot data. The return value must be a <code>data.frame</code>, and will be used as the layer data. A function can be created from a formula (e.g. <code>~ head(.x, 10)</code>).</p> |
| stat | <p>The statistical transformation to use on the data for this layer. When using a <code>geom_*()</code> function to construct a layer, the <code>stat</code> argument can be used to override the default coupling between geoms and stats. The <code>stat</code> argument accepts the following:</p> <ul style="list-style-type: none"> • A <code>Stat</code> ggproto subclass, for example <code>StatCount</code>. • A string naming the stat. To give the stat as a string, strip the function name of the <code>stat_</code> prefix. For example, to use <code>stat_count()</code>, give the stat as <code>"count"</code>. • For more information and other ways to specify the stat, see the layer stat documentation. |

position	A position adjustment to use on the data for this layer. This can be used in various ways, including to prevent overplotting and improving the display. The position argument accepts the following: • The result of calling a position function, such as <code>position_jitter()</code>. This method allows for passing extra arguments to the position. • A string naming the position adjustment. To give the position as a string, strip the function name of the position_ prefix. For example, to use <code>position_jitter()</code>, give the position as "jitter". • For more information and other ways to specify the position, see the layer position documentation.
...	Other arguments passed on to <code>layer()</code>'s params argument. These arguments broadly fall into one of 4 categories below. Notably, further arguments to the position argument, or aesthetics that are required can <i>not</i> be passed through ... Unknown arguments that are not part of the 4 categories below are ignored. • Static aesthetics that are not mapped to a scale, but are at a fixed value and apply to the layer as a whole. For example, <code>colour = "red"</code> or <code>linewidth = 3</code>. The geom's documentation has an Aesthetics section that lists the available options. The 'required' aesthetics cannot be passed on to the params. Please note that while passing unmapped aesthetics as vectors is technically possible, the order and required length is not guaranteed to be parallel to the input data. • When constructing a layer using a <code>stat_*()</code> function, the ... argument can be used to pass on parameters to the geom part of the layer. An example of this is <code>stat_density(geom = "area", outline.type = "both")</code>. The geom's documentation lists which parameters it can accept. • Inversely, when constructing a layer using a <code>geom_*()</code> function, the ... argument can be used to pass on parameters to the stat part of the layer. An example of this is <code>geom_area(stat = "density", adjust = 0.5)</code>. The stat's documentation lists which parameters it can accept. • The <code>key_glyph</code> argument of <code>layer()</code> may also be passed on through ... This can be one of the functions described as key glyphs, to change the display of the layer in the legend.
parse	If TRUE, the labels will be parsed into expressions and displayed as described in <code>?plotmath</code> .
nudge_x, nudge_y	Horizontal and vertical adjustment to nudge labels by. Useful for offsetting text from points, particularly on discrete scales.
check_overlap	If TRUE, text that overlaps previous text in the same layer will not be plotted. <code>check_overlap</code> happens at draw time and in the order of the data. Therefore data should be arranged by the label column before calling <code>geom_text()</code> . Note that this argument is not supported by <code>geom_label()</code> .
na.rm	If FALSE, the default, missing values are removed with a warning. If TRUE, missing values are silently removed.
show.legend	logical. Should this layer be included in the legends? NA, the default, includes if any aesthetics are mapped. FALSE never includes, and TRUE always includes. It can also be a named logical vector to finely select the aesthetics to display. To include legend keys for all levels, even when no data exists, use TRUE. If NA, all levels are shown in legend, but unobserved levels are omitted.
inherit.aes	If FALSE, overrides the default aesthetics, rather than combining with them. This is most useful for helper functions that define both data and aesthetics and shouldn't inherit behaviour from the default plot specification, e.g. annotation_borders() .

Details

The package will also take care of loading the StateFace font for PDF and other devices, but to use it with the on-screen ggplot2 device, you'll need to install the font on your system.

ggalt ships with a copy of the StateFace TTF font. You can run `show_stateface()` to get the filesystem location and then load the font manually from there.

A sample of the output from `geom_stateface()`:

See Also

Other StateFace operations: [load_stateface\(\)](#), [show_stateface\(\)](#)

Examples

```
## Not run:
library(ggplot2)
library(ggalt)

# Run show_stateface() to see the location of the TTF StateFace font
# You need to install it for it to work

set.seed(1492)
dat <- data.frame(state=state.abb,
                  x=sample(100, 50),
                  y=sample(100, 50),
                  col=sample(c("#b2182b", "#2166ac"), 50, replace=TRUE),
                  sz=sample(6:15, 50, replace=TRUE),
                  stringsAsFactors=FALSE)

gg <- ggplot(dat, aes(x=x, y=y))
gg <- gg + geom_stateface(aes(label=state, color=col, size=sz))
gg <- gg + scale_color_identity()
gg <- gg + scale_size_identity()
gg

## End(Not run)
```

geom_ubar

Uniform "bar" charts

Description

I've been using `geom_segment` more to make "bar" charts, setting `xend` to whatever `x` is and `yend` to 0. The bar widths remain constant without any tricks and you have granular control over the segment width. I decided it was time to make a `geom`.

Usage

```
geom_ubar(
  mapping = NULL,
  data = NULL,
  stat = "identity",
  position = "identity",
  ...,

```

```

na.rm = FALSE,
show.legend = NA,
inherit.aes = TRUE
)

```

Arguments

mapping	Set of aesthetic mappings created by aes() . If specified and <code>inherit.aes = TRUE</code> (the default), it is combined with the default mapping at the top level of the plot. You must supply mapping if there is no plot mapping.
data	The data to be displayed in this layer. There are three options: If <code>NULL</code>, the default, the data is inherited from the plot data as specified in the call to ggplot(). A <code>data.frame</code>, or other object, will override the plot data. All objects will be fortified to produce a data frame. See fortify() for which variables will be created. A function will be called with a single argument, the plot data. The return value must be a <code>data.frame</code>, and will be used as the layer data. A function can be created from a formula (e.g. <code>~ head(.x, 10)</code>).
stat	The statistical transformation to use on the data for this layer. When using a <code>geom_*()</code> function to construct a layer, the <code>stat</code> argument can be used to override the default coupling between geoms and stats. The <code>stat</code> argument accepts the following: • A <code>Stat</code> ggproto subclass, for example <code>StatCount</code>. • A string naming the stat. To give the stat as a string, strip the function name of the <code>stat_</code> prefix. For example, to use <code>stat_count()</code>, give the stat as <code>"count"</code>. • For more information and other ways to specify the stat, see the layer stat documentation.
position	A position adjustment to use on the data for this layer. This can be used in various ways, including to prevent overplotting and improving the display. The <code>position</code> argument accepts the following: • The result of calling a position function, such as <code>position_jitter()</code>. This method allows for passing extra arguments to the position. • A string naming the position adjustment. To give the position as a string, strip the function name of the <code>position_</code> prefix. For example, to use <code>position_jitter()</code>, give the position as <code>"jitter"</code>. • For more information and other ways to specify the position, see the layer position documentation.
...	other arguments passed on to <code>layer</code> . These are often aesthetics, used to set an aesthetic to a fixed value, like <code>color = "red"</code> or <code>size = 3</code> . They may also be parameters to the paired geom/stat.
na.rm	If <code>FALSE</code> (the default), removes missing values with a warning. If <code>TRUE</code> silently removes missing values.
show.legend	logical. Should this layer be included in the legends? <code>NA</code> , the default, includes if any aesthetics are mapped. <code>FALSE</code> never includes, and <code>TRUE</code> always includes. It can also be a named logical vector to finely select the aesthetics to display. To include legend keys for all levels, even when no data exists, use <code>TRUE</code> . If <code>NA</code> , all levels are shown in legend, but unobserved levels are omitted.

`inherit.aes` If FALSE, overrides the default aesthetics, rather than combining with them. This is most useful for helper functions that define both data and aesthetics and shouldn't inherit behaviour from the default plot specification, e.g. `annotation_borders()`.

Aesthetics

'geom_ubar' understands the following aesthetics (required aesthetics are in bold):

- x
- y
- alpha
- colour
- group
- linetype
- size

Examples

```
library(ggplot2)

data(economics)
ggplot(economics, aes(date, uempmed)) +
  geom_ubar()
```

<code>geom_xspline</code>	<i>Connect control points/observations with an X-spline</i>
---------------------------	---

Description

Draw an X-spline, a curve drawn relative to control points/observations. Patterned after `geom_line` in that it orders the points by x first before computing the splines.

Usage

```
geom_xspline(
  mapping = NULL,
  data = NULL,
  stat = "xspline",
  position = "identity",
  na.rm = TRUE,
  show.legend = NA,
  inherit.aes = TRUE,
  spline_shape = -0.25,
  open = TRUE,
  rep_ends = TRUE,
  ...
)

stat_xspline(
  mapping = NULL,
```

```

data = NULL,
geom = "line",
position = "identity",
na.rm = TRUE,
show.legend = NA,
inherit.aes = TRUE,
spline_shape = -0.25,
open = TRUE,
rep_ends = TRUE,
...
)

```

Arguments

mapping	Set of aesthetic mappings created by aes() . If specified and <code>inherit.aes = TRUE</code> (the default), it is combined with the default mapping at the top level of the plot. You must supply mapping if there is no plot mapping.
data	The data to be displayed in this layer. There are three options: If <code>NULL</code>, the default, the data is inherited from the plot data as specified in the call to ggplot(). A <code>data.frame</code>, or other object, will override the plot data. All objects will be fortified to produce a data frame. See fortify() for which variables will be created. A function will be called with a single argument, the plot data. The return value must be a <code>data.frame</code>, and will be used as the layer data. A function can be created from a formula (e.g. <code>~ head(.x, 10)</code>).
position	A position adjustment to use on the data for this layer. This can be used in various ways, including to prevent overplotting and improving the display. The position argument accepts the following: • The result of calling a position function, such as position_jitter(). This method allows for passing extra arguments to the position. • A string naming the position adjustment. To give the position as a string, strip the function name of the <code>position_</code> prefix. For example, to use position_jitter(), give the position as "jitter". • For more information and other ways to specify the position, see the layer position documentation.
na.rm	If <code>FALSE</code> , the default, missing values are removed with a warning. If <code>TRUE</code> , missing values are silently removed.
show.legend	logical. Should this layer be included in the legends? <code>NA</code> , the default, includes if any aesthetics are mapped. <code>FALSE</code> never includes, and <code>TRUE</code> always includes. It can also be a named logical vector to finely select the aesthetics to display. To include legend keys for all levels, even when no data exists, use <code>TRUE</code> . If <code>NA</code> , all levels are shown in legend, but unobserved levels are omitted.
inherit.aes	If <code>FALSE</code> , overrides the default aesthetics, rather than combining with them. This is most useful for helper functions that define both data and aesthetics and shouldn't inherit behaviour from the default plot specification, e.g. annotation_borders() .
spline_shape	A numeric vector of values between -1 and 1, which control the shape of the spline relative to the control points.
open	A logical value indicating whether the spline is an open or a closed shape.

rep_ends	For open X-splines, a logical value indicating whether the first and last control points should be replicated for drawing the curve. Ignored for closed X-splines.
...	Other arguments passed on to <code>layer()</code>'s <code>params</code> argument. These arguments broadly fall into one of 4 categories below. Notably, further arguments to the <code>position</code> argument, or aesthetics that are required can <i>not</i> be passed through ... Unknown arguments that are not part of the 4 categories below are ignored. • Static aesthetics that are not mapped to a scale, but are at a fixed value and apply to the layer as a whole. For example, <code>colour = "red"</code> or <code>linewidth = 3</code>. The geom's documentation has an Aesthetics section that lists the available options. The 'required' aesthetics cannot be passed on to the <code>params</code>. Please note that while passing unmapped aesthetics as vectors is technically possible, the order and required length is not guaranteed to be parallel to the input data. • When constructing a layer using a <code>stat_*()</code> function, the <code>...</code> argument can be used to pass on parameters to the geom part of the layer. An example of this is <code>stat_density(geom = "area", outline.type = "both")</code>. The geom's documentation lists which parameters it can accept. • Inversely, when constructing a layer using a <code>geom_*()</code> function, the <code>...</code> argument can be used to pass on parameters to the stat part of the layer. An example of this is <code>geom_area(stat = "density", adjust = 0.5)</code>. The stat's documentation lists which parameters it can accept. • The <code>key_glyph</code> argument of <code>layer()</code> may also be passed on through ... This can be one of the functions described as key glyphs, to change the display of the layer in the legend.
geom, stat	Use to override the default connection between <code>geom_xspline</code> and <code>stat_xspline</code> .

Details

A sample of the output from `geom_xspline()`:

An X-spline is a line drawn relative to control points. For each control point, the line may pass through (interpolate) the control point or it may only approach (approximate) the control point; the behaviour is determined by a shape parameter for each control point.

If the shape parameter is greater than zero, the spline approximates the control points (and is very similar to a cubic B-spline when the shape is 1). If the shape parameter is less than zero, the spline interpolates the control points (and is very similar to a Catmull-Rom spline when the shape is -1). If the shape parameter is 0, the spline forms a sharp corner at that control point.

For open X-splines, the start and end control points must have a shape of 0 (and non-zero values are silently converted to zero).

For open X-splines, by default the start and end control points are replicated before the curve is drawn. A curve is drawn between (interpolating or approximating) the second and third of each set of four control points, so this default behaviour ensures that the resulting curve starts at the first control point you have specified and ends at the last control point. The default behaviour can be turned off via the `repEnds` argument.

Aesthetics

`geom_xspline` understands the following aesthetics (required aesthetics are in bold):

- x
- y

- alpha
- color
- linetype
- size

Computed variables

- x
- y

References

Blanc, C. and Schlick, C. (1995), "X-splines : A Spline Model Designed for the End User", in *Proceedings of SIGGRAPH 95*, pp. 377-386. <https://dept-info.labri.fr/~schlick/DOC/sig1.html>

See Also

[geom_line](#): Connect observations (x order); [geom_path](#): Connect observations; [geom_polygon](#): Filled paths (polygons); [geom_segment](#): Line segments; [xspline](#); [grid.xspline](#)

Other xspline implementations: [geom_xspline2\(\)](#)

Examples

```
set.seed(1492)
dat <- data.frame(x=c(1:10, 1:10, 1:10),
                  y=c(sample(15:30, 10), 2*sample(15:30, 10),
                      3*sample(15:30, 10)),
                  group=factor(c(rep(1, 10), rep(2, 10), rep(3, 10))))

ggplot(dat, aes(x, y, group=group, color=group)) +
  geom_point() +
  geom_line()

ggplot(dat, aes(x, y, group=group, color=factor(group))) +
  geom_point() +
  geom_line() +
  geom_smooth(se=FALSE, linetype="dashed", size=0.5)

ggplot(dat, aes(x, y, group=group, color=factor(group))) +
  geom_point(color="black") +
  geom_smooth(se=FALSE, linetype="dashed", size=0.5) +
  geom_xspline(size=0.5)

ggplot(dat, aes(x, y, group=group, color=factor(group))) +
  geom_point(color="black") +
  geom_smooth(se=FALSE, linetype="dashed", size=0.5) +
  geom_xspline(spline_shape=-0.4, size=0.5)

ggplot(dat, aes(x, y, group=group, color=factor(group))) +
  geom_point(color="black") +
  geom_smooth(se=FALSE, linetype="dashed", size=0.5) +
  geom_xspline(spline_shape=0.4, size=0.5)
```

```

ggplot(dat, aes(x, y, group=group, color=factor(group))) +
  geom_point(color="black") +
  geom_smooth(se=FALSE, linetype="dashed", size=0.5) +
  geom_xspline(spline_shape=1, size=0.5)

ggplot(dat, aes(x, y, group=group, color=factor(group))) +
  geom_point(color="black") +
  geom_smooth(se=FALSE, linetype="dashed", size=0.5) +
  geom_xspline(spline_shape=0, size=0.5)

ggplot(dat, aes(x, y, group=group, color=factor(group))) +
  geom_point(color="black") +
  geom_smooth(se=FALSE, linetype="dashed", size=0.5) +
  geom_xspline(spline_shape=-1, size=0.5)

```

geom_xspline2	<i>Alternative implemenation for connecting control points/observations with an X-spline</i>
---------------	--

Description

Alternative implemenation for connecting control points/observations with an X-spline

Usage

```

geom_xspline2(
  mapping = NULL,
  data = NULL,
  stat = "identity",
  position = "identity",
  na.rm = FALSE,
  show.legend = NA,
  inherit.aes = TRUE,
  ...
)

```

Arguments

mapping	Set of aesthetic mappings created by aes() . If specified and <code>inherit.aes = TRUE</code> (the default), it is combined with the default mapping at the top level of the plot. You must supply mapping if there is no plot mapping.
data	The data to be displayed in this layer. There are three options: If <code>NULL</code>, the default, the data is inherited from the plot data as specified in the call to ggplot(). A <code>data.frame</code>, or other object, will override the plot data. All objects will be fortified to produce a data frame. See fortify() for which variables will be created. A function will be called with a single argument, the plot data. The return value must be a <code>data.frame</code>, and will be used as the layer data. A function can be created from a formula (e.g. <code>~ head(.x, 10)</code>).

stat	Other arguments passed to layer
position	A position adjustment to use on the data for this layer. This can be used in various ways, including to prevent overplotting and improving the display. The position argument accepts the following: • The result of calling a position function, such as <code>position_jitter()</code>. This method allows for passing extra arguments to the position. • A string naming the position adjustment. To give the position as a string, strip the function name of the <code>position_</code> prefix. For example, to use <code>position_jitter()</code>, give the position as "jitter". • For more information and other ways to specify the position, see the layer position documentation.
na.rm	If FALSE, the default, missing values are removed with a warning. If TRUE, missing values are silently removed.
show.legend	logical. Should this layer be included in the legends? NA, the default, includes if any aesthetics are mapped. FALSE never includes, and TRUE always includes. It can also be a named logical vector to finely select the aesthetics to display. To include legend keys for all levels, even when no data exists, use TRUE. If NA, all levels are shown in legend, but unobserved levels are omitted.
inherit.aes	If FALSE, overrides the default aesthetics, rather than combining with them. This is most useful for helper functions that define both data and aesthetics and shouldn't inherit behaviour from the default plot specification, e.g. annotation_borders() .
...	Other arguments passed on to layer()'s <code>params</code> argument. These arguments broadly fall into one of 4 categories below. Notably, further arguments to the position argument, or aesthetics that are required can <i>not</i> be passed through ... Unknown arguments that are not part of the 4 categories below are ignored. • Static aesthetics that are not mapped to a scale, but are at a fixed value and apply to the layer as a whole. For example, <code>colour = "red"</code> or <code>linewidth = 3</code>. The geom's documentation has an Aesthetics section that lists the available options. The 'required' aesthetics cannot be passed on to the <code>params</code>. Please note that while passing unmapped aesthetics as vectors is technically possible, the order and required length is not guaranteed to be parallel to the input data. • When constructing a layer using a <code>stat_*()</code> function, the ... argument can be used to pass on parameters to the geom part of the layer. An example of this is <code>stat_density(geom = "area", outline.type = "both")</code>. The geom's documentation lists which parameters it can accept. • Inversely, when constructing a layer using a <code>geom_*()</code> function, the ... argument can be used to pass on parameters to the stat part of the layer. An example of this is <code>geom_area(stat = "density", adjust = 0.5)</code>. The stat's documentation lists which parameters it can accept. • The <code>key_glyph</code> argument of layer() may also be passed on through ... This can be one of the functions described as key glyphs, to change the display of the layer in the legend.

Value

creates a spline curve

Author(s)

Ben Bolker

See Also

Other xspline implementations: [geom_xspline\(\)](#)

load_stateface

Load stateface font

Description

Makes the ProPublica StateFace font available to PDF, PostScript, et. al. devices.

Usage

```
load_stateface()
```

See Also

Other StateFace operations: [geom_stateface\(\)](#), [show_stateface\(\)](#)

position_dodgev

Vertically dodge position

Description

Vertically dodge position

Usage

```
position_dodgev(height = NULL)
```

Arguments

height numeric, height of vertical dodge, Default: NULL

Note

position-dodgev(): unmodified from lionel-/ggstance/R/position-dodgev.R 73f521384ae8ea277db5f7d5a2854004aa18f9

Author(s)

@ggstance authors

Examples

```
if(interactive()){

dat <- data.frame(
  trt = c(LETTERS[1:5], "D"),
  l = c(20, 40, 10, 30, 50, 40),
  r = c(70, 50, 30, 60, 80, 70)
)

ggplot(dat, aes(y=trt, x=l, xend=r)) +
  geom_dumbbell(size=3, color="#e3e2e1",
               colour_x = "#5b8124", colour_xend = "#bad744",
               dot_guide=TRUE, dot_guide_size=0.25,
               position=position_dodgev(height=0.8)) +
  labs(x=NULL, y=NULL, title="ggplot2 geom_dumbbell with dot guide") +
  theme_minimal() +
  theme(panel.grid.major.x=element_line(size=0.05))

}
```

show_stateface	Show location of StateFace font
----------------	---------------------------------

Description

Displays the path to the StateFace font. For the font to work in the on-screen plot device for ggplot2, you need to install the font on your system

Usage

```
show_stateface()
```

See Also

Other StateFace operations: [geom_stateface\(\)](#), [load_stateface\(\)](#)

StatAsh	Geom Proto
---------	------------

Description

- Geom Proto
- Geom Proto
- Geom Proto
- Geom Proto

References

<https://groups.google.com/forum/?fromgroups=#!topic/ggplot2/9cFWHaH1CPs>

stat_ash	<i>Compute and display a univariate averaged shifted histogram (polynomial kernel)</i>
----------	--

Description

See [bin1](#) & [ash1](#) for more information.

Usage

```
stat_ash(
  mapping = NULL,
  data = NULL,
  geom = "area",
  position = "stack",
  ab = NULL,
  nbin = 50,
  m = 5,
  kopt = c(2, 2),
  na.rm = FALSE,
  show.legend = NA,
  inherit.aes = TRUE,
  ...
)
```

Arguments

mapping	Set of aesthetic mappings created by aes() . If specified and <code>inherit.aes = TRUE</code> (the default), it is combined with the default mapping at the top level of the plot. You must supply mapping if there is no plot mapping.
data	The data to be displayed in this layer. There are three options: If <code>NULL</code>, the default, the data is inherited from the plot data as specified in the call to ggplot(). A <code>data.frame</code>, or other object, will override the plot data. All objects will be fortified to produce a data frame. See fortify() for which variables will be created. A function will be called with a single argument, the plot data. The return value must be a <code>data.frame</code>, and will be used as the layer data. A function can be created from a formula (e.g. <code>~ head(.x, 10)</code>).
geom	Use to override the default Geom
position	A position adjustment to use on the data for this layer. This can be used in various ways, including to prevent overplotting and improving the display. The position argument accepts the following: • The result of calling a position function, such as position_jitter(). This method allows for passing extra arguments to the position. • A string naming the position adjustment. To give the position as a string, strip the function name of the <code>position_</code> prefix. For example, to use position_jitter(), give the position as "jitter".

	For more information and other ways to specify the position, see the layer position documentation.
ab	half-open interval for bins $[a,b)$. If no value is specified, the range of x is stretched by 5% at each end and used the interval.
nbin	number of bins desired. Default 50.
m	integer smoothing parameter; Default 5.
kopt	vector of length 2 specifying the kernel, which is proportional to $(1 - \text{abs}(i/m)^{\text{kopt}(1)})^{\text{kopt}(2)}$; (2,2)=biweight (default); (0,0)=uniform; (1,0)=triangle; (2,1)=Epanechnikov; (2,3)=triweight.
na.rm	If FALSE, the default, missing values are removed with a warning. If TRUE, missing values are silently removed.
show.legend	logical. Should this layer be included in the legends? NA, the default, includes if any aesthetics are mapped. FALSE never includes, and TRUE always includes. It can also be a named logical vector to finely select the aesthetics to display. To include legend keys for all levels, even when no data exists, use TRUE. If NA, all levels are shown in legend, but unobserved levels are omitted.
inherit.aes	If FALSE, overrides the default aesthetics, rather than combining with them. This is most useful for helper functions that define both data and aesthetics and shouldn't inherit behaviour from the default plot specification, e.g. annotation_borders() .
...	Other arguments passed on to layer()'s params argument. These arguments broadly fall into one of 4 categories below. Notably, further arguments to the position argument, or aesthetics that are required can <i>not</i> be passed through Unknown arguments that are not part of the 4 categories below are ignored. - Static aesthetics that are not mapped to a scale, but are at a fixed value and apply to the layer as a whole. For example, <code>colour = "red"</code> or <code>linewidth = 3</code>. The geom's documentation has an Aesthetics section that lists the available options. The 'required' aesthetics cannot be passed on to the params. Please note that while passing unmapped aesthetics as vectors is technically possible, the order and required length is not guaranteed to be parallel to the input data. - When constructing a layer using a <code>stat_*()</code> function, the ... argument can be used to pass on parameters to the geom part of the layer. An example of this is <code>stat_density(geom = "area", outline.type = "both")</code>. The geom's documentation lists which parameters it can accept. - Inversely, when constructing a layer using a <code>geom_*()</code> function, the ... argument can be used to pass on parameters to the stat part of the layer. An example of this is <code>geom_area(stat = "density", adjust = 0.5)</code>. The stat's documentation lists which parameters it can accept. - The <code>key_glyph</code> argument of layer() may also be passed on through This can be one of the functions described as key glyphs, to change the display of the layer in the legend.

Details

A sample of the output from `stat_ash()`:

Aesthetics

`geom_ash` understands the following aesthetics (required aesthetics are in bold):

- x
- alpha
- color
- fill
- linetype
- size

Computed variables

density ash density estimate

References

David Scott (1992), *"Multivariate Density Estimation,"* John Wiley, (chapter 5 in particular).

B. W. Silverman (1986), *"Density Estimation for Statistics and Data Analysis,"* Chapman & Hall.

Examples

```
# compare
library(gridExtra)
set.seed(1492)
dat <- data.frame(x=rnorm(100))
grid.arrange(ggplot(dat, aes(x)) + stat_ash(),
              ggplot(dat, aes(x)) + stat_bkde(),
              ggplot(dat, aes(x)) + stat_density(),
              nrow=3)

cols <- RColorBrewer::brewer.pal(3, "Dark2")
ggplot(dat, aes(x)) +
  stat_ash(alpha=1/2, fill=cols[3]) +
  stat_bkde(alpha=1/2, fill=cols[2]) +
  stat_density(alpha=1/2, fill=cols[1]) +
  geom_rug() +
  labs(x=NULL, y="density/estimate") +
  scale_x_continuous(expand=c(0,0)) +
  theme_bw() +
  theme(panel.grid=element_blank()) +
  theme(panel.border=element_blank())
```

stat_stepribbon

Step ribbon statistic

Description

Provides stairstep values for ribbon plots

Usage

```
stat_stepribbon(
  mapping = NULL,
  data = NULL,
  geom = "ribbon",
  position = "identity",
  na.rm = FALSE,
  show.legend = NA,
  inherit.aes = TRUE,
  direction = "hv",
  ...
)
```

Arguments

mapping	Set of aesthetic mappings created by aes() . If specified and <code>inherit.aes = TRUE</code> (the default), it is combined with the default mapping at the top level of the plot. You must supply mapping if there is no plot mapping.
data	The data to be displayed in this layer. There are three options: If <code>NULL</code>, the default, the data is inherited from the plot data as specified in the call to ggplot(). A <code>data.frame</code>, or other object, will override the plot data. All objects will be fortified to produce a data frame. See fortify() for which variables will be created. A function will be called with a single argument, the plot data. The return value must be a <code>data.frame</code>, and will be used as the layer data. A function can be created from a formula (e.g. <code>~ head(.x, 10)</code>).
geom	which geom to use; defaults to "ribbon"
position	A position adjustment to use on the data for this layer. This can be used in various ways, including to prevent overplotting and improving the display. The <code>position</code> argument accepts the following: • The result of calling a position function, such as position_jitter(). This method allows for passing extra arguments to the position. • A string naming the position adjustment. To give the position as a string, strip the function name of the <code>position_</code> prefix. For example, to use position_jitter(), give the position as "jitter". • For more information and other ways to specify the position, see the layer position documentation.
na.rm	If <code>FALSE</code> , the default, missing values are removed with a warning. If <code>TRUE</code> , missing values are silently removed.
show.legend	logical. Should this layer be included in the legends? <code>NA</code> , the default, includes if any aesthetics are mapped. <code>FALSE</code> never includes, and <code>TRUE</code> always includes. It can also be a named logical vector to finely select the aesthetics to display. To include legend keys for all levels, even when no data exists, use <code>TRUE</code> . If <code>NA</code> , all levels are shown in legend, but unobserved levels are omitted.
inherit.aes	If <code>FALSE</code> , overrides the default aesthetics, rather than combining with them. This is most useful for helper functions that define both data and aesthetics and shouldn't inherit behaviour from the default plot specification, e.g. annotation_borders() .
direction	hv for horizontal-vertical steps, vh for vertical-horizontal steps

...

Other arguments passed on to `layer()`'s `params` argument. These arguments broadly fall into one of 4 categories below. Notably, further arguments to the `position` argument, or aesthetics that are required can *not* be passed through Unknown arguments that are not part of the 4 categories below are ignored.

- Static aesthetics that are not mapped to a scale, but are at a fixed value and apply to the layer as a whole. For example, `colour = "red"` or `linewidth = 3`. The geom's documentation has an **Aesthetics** section that lists the available options. The 'required' aesthetics cannot be passed on to the `params`. Please note that while passing unmapped aesthetics as vectors is technically possible, the order and required length is not guaranteed to be parallel to the input data.
- When constructing a layer using a `stat_*()` function, the ... argument can be used to pass on parameters to the geom part of the layer. An example of this is `stat_density(geom = "area", outline.type = "both")`. The geom's documentation lists which parameters it can accept.
- Inversely, when constructing a layer using a `geom_*()` function, the ... argument can be used to pass on parameters to the stat part of the layer. An example of this is `geom_area(stat = "density", adjust = 0.5)`. The stat's documentation lists which parameters it can accept.
- The `key_glyph` argument of `layer()` may also be passed on through This can be one of the functions described as [key glyphs](#), to change the display of the layer in the legend.

References

<https://groups.google.com/forum/?fromgroups=#!topic/ggplot2/9cFWHaH1CPs>

Examples

```
x <- 1:10
df <- data.frame(x=x, y=x+10, ymin=x+7, ymax=x+12)

gg <- ggplot(df, aes(x, y))
gg <- gg + geom_ribbon(aes(ymin=ymin, ymax=ymax),
                      stat="stepribbon", fill="#b2b2b2")
gg <- gg + geom_step(color="#2b2b2b")
gg

gg <- ggplot(df, aes(x, y))
gg <- gg + geom_ribbon(aes(ymin=ymin, ymax=ymax),
                      stat="stepribbon", fill="#b2b2b2",
                      direction="hv")
gg <- gg + geom_step(color="#2b2b2b")
gg
```

Index

* StateFace operations

geom_stateface, 27
load_stateface, 37
show_stateface, 38

* datasets

GeomTicks, 8
position_dodgev, 37
StatAsh, 38

* internal

absoluteGrob, 3
geom_horizon, 21
ggalt-package, 2
StatAsh, 38

* xspline implementations

geom_xspline, 31
geom_xspline2, 35

absoluteGrob, 3

aes(), 9, 12, 15, 18, 23, 25, 27, 30, 32, 35, 39, 42

alpha, 19, 24

annotate_textp, 4

annotation_borders(), 10, 13, 16, 19, 23, 26, 28, 31, 32, 36, 40, 42

annotation_ticks, 5

ash1, 39

bin1, 39

bkde, 10

bkde2D, 13, 14

byte_format, 7

bytes (byte_format), 7

colour, 19, 24

fill, 24

fortify, 16

fortify(), 9, 13, 15, 18, 23, 25, 27, 30, 32, 35, 39, 42

fortify.table, 8

Gb (byte_format), 7

geom_bkde, 8

geom_bkde2d, 12

geom_cartogram, 15

geom_contour, 14

geom_dumbbell, 18

geom_encircle, 20

geom_freqpoly, 11

geom_histogram, 11

geom_horizon, 21

geom_line, 34

geom_lollipop, 22

geom_path, 34

geom_polygon, 34

geom_segment, 34

geom_spikelines, 24

geom_stateface, 27, 37, 38

geom_ubar, 29

geom_violin, 11

geom_xspline, 31, 37

geom_xspline2, 34, 35

GeomBkde (StatAsh), 38

GeomBkde2d (StatAsh), 38

GeomCartogram (GeomTicks), 8

GeomDumbbell (StatAsh), 38

GeomEncircle (StatAsh), 38

GeomHorizon (geom_horizon), 21

GeomLollipop (StatAsh), 38

GeomSpikelines (StatAsh), 38

GeomStateface (StatAsh), 38

GeomTicks, 8

GeomUbar (StatAsh), 38

GeomXspline (StatAsh), 38

GeomXspline2 (StatAsh), 38

ggalt (ggalt-package), 2

ggalt-package, 2

ggplot(), 9, 12, 15, 18, 23, 25, 27, 30, 32, 35, 39, 42

grid.xspline, 34

grid::arrow(), 26

grid::unit(), 5

group, 19, 24

Kb (byte_format), 7

key glyphs, 10, 14, 16, 26, 28, 33, 36, 40, 43

layer, 19, 23, 36

layer position, [10](#), [13](#), [25](#), [28](#), [30](#), [32](#), [36](#),
[40](#), [42](#)
layer stat, [13](#), [16](#), [25](#), [27](#), [30](#)
layer(), [10](#), [14](#), [16](#), [25](#), [26](#), [28](#), [33](#), [36](#), [40](#), [43](#)
linetype, [19](#)
linewidth, [19](#)
load_stateface, [29](#), [37](#), [38](#)

Mb (byte_format), [7](#)

position_dodgev, [37](#)
PositionDodgev (position_dodgev), [37](#)

shape, [24](#)
show_stateface, [29](#), [37](#), [38](#)
size, [24](#)
stat_ash, [39](#)
stat_bkde (geom_bkde), [8](#)
stat_bkde2d (geom_bkde2d), [12](#)
stat_contour, [14](#)
stat_horizon (geom_horizon), [21](#)
stat_stepribbon, [41](#)
stat_sum, [14](#)
stat_xspline (geom_xspline), [31](#)
StatAsh, [38](#)
StatBkde (StatAsh), [38](#)
StatBkde2d (StatAsh), [38](#)
StatHorizon (geom_horizon), [21](#)
StatStepribbon (StatAsh), [38](#)
StatXspline (StatAsh), [38](#)

x, [19](#), [24](#)
xend, [19](#)
xspline, [34](#)

y, [19](#), [24](#)
yend, [19](#)